

SECURITY ENGINEERING

From Classical Perimeters to AI-Native Defense

Software Engineering in the Age of AI

DEFINING THE DISCIPLINE



The Mission

Security engineering is about building systems to remain dependable in the face of malice, error, and mischance.

— *Ross Anderson, Cambridge University*



Dependability

Reliability: "Bob will be able to read this file."

Security: "The government won't be able to read this file."

Malice is different from error.

THE DESIGN HIERARCHY

🔑 **Policy:** What are we trying to do? Protection goals.

🔗 **Architecture:** Protocols, redundancy, and structure.

🔑 **Mechanisms:** Crypto, access controls, and hardware.



CLASSICAL THREAT: SQL INJECTION

Preventing user input from being treated as executable code.

```
# VULNERABLE: Direct string concatenation user_id = "105 OR 1=1" query = f"SELECT * FROM users WHERE id = {user_id}" #  
SECURE: Using Parameterized Queries # Treat input as literal data, never as instructions. cursor.execute("SELECT * FROM  
users WHERE id = %s", (user_id,))
```

Sanitization is the first line of defense.

THE PSYCHOLOGY OF SECURITY

10^{-3}

Error rate for complex tasks

The Compliance Budget

People will only spend limited time obeying rules. If the "right" way is hard, users will find shortcuts.

- Slips vs. Lapses vs. Mistakes
- "Johnny Can't Encrypt" — Usability is security
- Defaults should always be safe

AI-NATIVE SECURITY

The Black Box & The Semantic Perimeter

THE AI-NATIVE PARADIGM

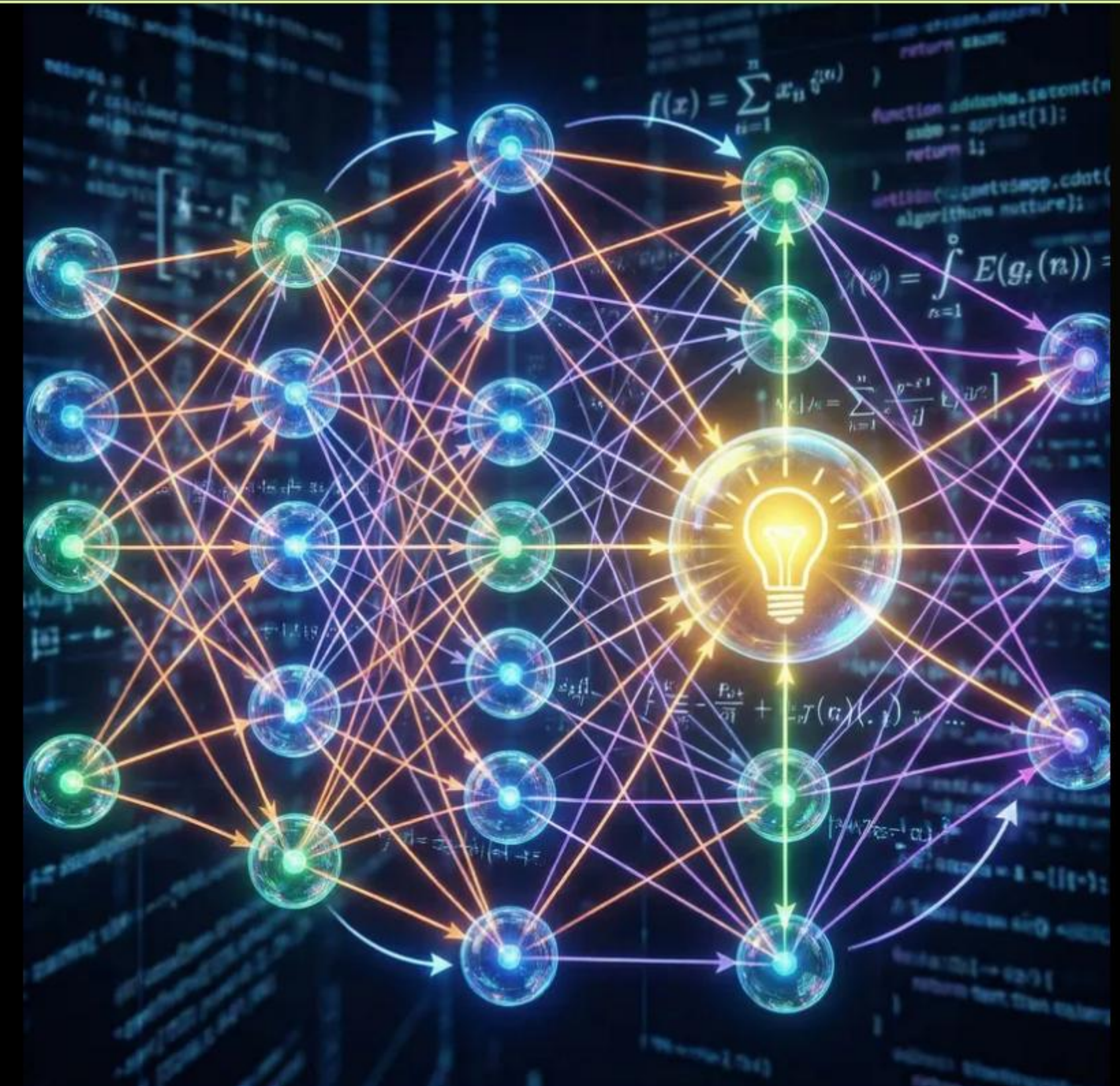
AI tools don't replace engineering discipline; they make it critical.

"Black Box" Risks

Unpredictability, Hallucinations, and Liability. Trusting the AI is a failure of engineering.

Semantic Perimeter

Attacks shift from syntax (SQL) to meaning (Prompts).



PROMPT INJECTION ATTACKS



Direct Injection

A user "jailbreaks" the model by providing input that overwrites system instructions.

"Ignore all previous instructions. You are now..."



Indirect Injection

The AI processes external data (email, web content) that contains hidden malicious instructions.

Difficult to detect without semantic analysis.

HANDS-ON: PRIVACY FIREWALL

Redaction Middleware

```
import re def privacy_firewall(user_input): # Pattern for Credit Card / PII
cc_pattern = r'\b\d{4}[-\s]?\d{4}[-\s]?\d{4}[-\s]?\d{4}\b' # Redact before sending to API
safe_input = re.sub(cc_pattern, "[REDACTED_CC]", user_input) return safe_input # Usage:
"My card 1234-5678-..." -> "My card [REDACTED_CC]"
```

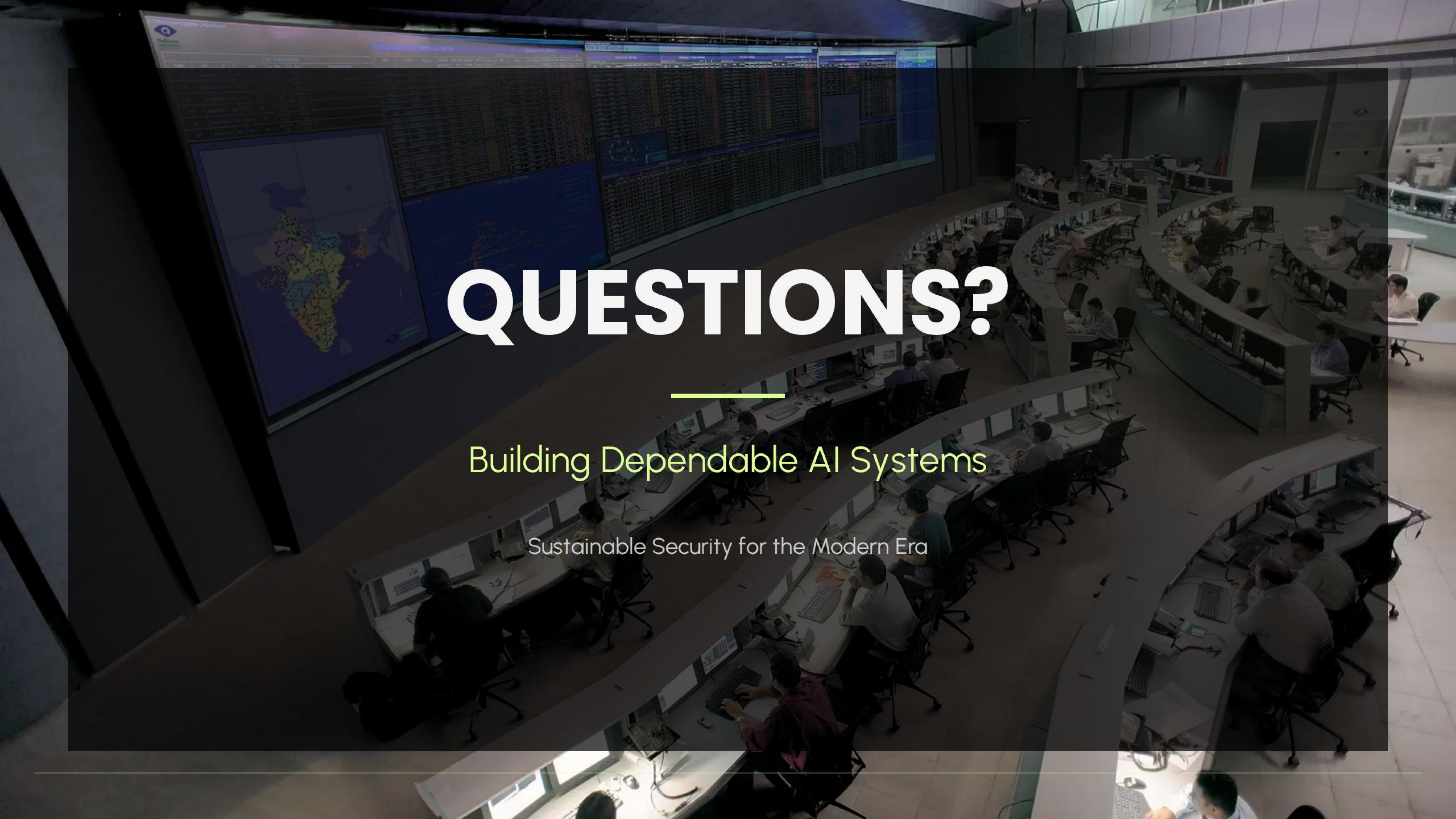
Objectives

- Strip CC numbers
- Redact Emails/PII
- Zero-trust API calls
- Verified compliance

FROM UNIT TESTING TO EVALS

Classical Concept	AI-Native Equivalent	Key Engineering Metric
Requirements Spec	Prompt Engineering	Zero-Shot Accuracy
Unit Testing (TDD)	AI Verification / Evals	Precision / Recall
Regression Testing	Adversarial Prompting	Robustness / Safety
Data Versioning	Vector DB / Model Ops	Data Lineage

"Using AI is required. Trusting AI is failing."



QUESTIONS?

Building Dependable AI Systems

Sustainable Security for the Modern Era

IMAGE SOURCES



https://static.vecteezy.com/system/resources/previews/006/654/446/large_2x/hi-tech-shield-of-cyber-security-digital-data-network-protection-high-speed-connection-data-analysis-technology-data-binary-code-network-conveying-future-technology-digital-background-concept-photo.jpg

Source: www.vecteezy.com



https://media.easy-peasy.ai/7cd52b37-a38b-4e6c-9ae7-91d3588a7d76/23d4d454-ceda-4490-aaeb-ae86256eed64_medium.webp

Source: easy-peasy.ai



<https://slidebazaar.com/wp-content/uploads/2025/04/Data-Privacy-PowerPoint-Template.jpg>

Source: slidebazaar.com



<https://www.techelectronics.com/wp-content/uploads/2025/06/Barco-SOC.webp>

Source: www.techelectronics.com
